

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №6
по курсу «Алгоритмы и структуры данных»
Тема: Хеширование. Хеш-таблицы
Вариант 20

Выполнил:
Степанова Е.А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
<u>Задача №1. Множество [N баллов]</u>	
<u>Задача №2. Телефонная книга [N баллов]</u>	
7	
<u>Задача №5. Выборы США [N баллов]</u>	
13	
<u>Задача №6. Фибоначчи возвращается [N баллов]</u>	16
<u>Задача №8. Фибоначчи возвращается [N баллов]</u>	19
Вывод	22

Задачи по варианту

Задача №1. Множество [N баллов]

Реализуйте множество с операциями «добавление ключа», «удаление ключа», «проверка существования ключа».

- **Формат входного файла (input.txt).** В первой строке входного файла находится строго положительное целое число операций N , не превышающее $5 \cdot 10^5$. В каждой из последующих N строк находится одна из следующих операций:
 - A x – добавить элемент x в множество. Если элемент уже есть в множестве, то ничего делать не надо.
 - D x – удалить элемент x . Если элемента x нет, то ничего делать не надо.
 - ? x – если ключ x есть в множестве, выведите «Y», если нет, то выведите «N».

Аргументы указанных выше операций – **целые числа**, не превышающие по модулю 10^{18} .

- **Формат выходного файла (output.txt).** Выведите последовательно результат выполнения всех операций «?». Следуйте формату выходного файла из примера.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
8	Y
A 2	N
A 5	N
A 3	
? 2	
? 4	
A 2	
D 2	
? 2	

- **Примечание.**

Эту задачу можно решить совершенно разными способами, включая использование различных средств стандартных библиотек (правда, не всех - в стандартных библиотеках некоторых языков программирования используются слишком предсказуемые методы хеширования). Именно по этой причине ее разумно использовать для проверки реализаций хеш-таблиц, которые понадобятся в следующих задачах этой работы.

```

from lab_6.utils import read_f, write_f

s = set()
answer = []

def add_A(data):
    s.add(data)

def find(data):
    if data in s:
        answer.append('Y')
    else:
        answer.append('N')

def del_D(data):
    s.remove(data)

def do_task(n, a):
    for i in range(n):
        command, data = a[i].split()
        if command == 'A':
            add_A(data)
        elif command == 'D':
            del_D(data)
        elif command == '?':
            find(data)
    return answer

if __name__ == "__main__":
    data = read_f(1)
    n = int(data[0])
    data.pop(0)
    res = do_task(n, data)
    write_f(1, *res)

```

Данный код выполняет следующие функции:

Импортирует две функции из модуля `utils`: `read_f` и `write_f`. Эти функции используются для чтения данных из файла и записи результатов в файл соответственно.

Создаёт пустой набор `s` и пустой список `answer`.

Определяет три функции: `add_A`, `find` и `del_D`. Функция `add_A` добавляет элемент в набор `s`. Функция `find` проверяет, есть ли элемент в наборе `s`, и добавляет «Y» или «N» в список `answer` в зависимости от результата проверки. Функция `del_D` удаляет элемент из набора `s`.

Определяет функцию `do_task`, которая принимает два аргумента: количество команд `n` и список команд `a`. В этой функции происходит обработка каждой команды. Если команда — «A», то вызывается функция `add_A`. Если команда — «D», то вызывается функция `del_D`. Если команда — «?», то вызывается функция `find`. После обработки всех команд возвращается список `answer`, который содержит результаты выполнения каждой команды.

В основной части кода считывается первая строка из файла с помощью функции `read_f`, преобразуется в число `n`, а затем удаляются данные из строки. Далее вызывается функция `do_task` с аргументами `n` и оставшимися данными из файла. Результаты выполнения команд сохраняются в списке `res`. Затем этот список записывается в файл с помощью функции `write_f`.

input.txt ×			output.txt ×		
1	8	✓	1	Y	
2	A 2		2	N	
3	A 5		3	N	
4	A 3		4		
5	? 2				
6	? 4				
7	A 2				
8	D 2				
9	? 2				

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.00011709536314e-06	64
Пример из задачи	1.0299991117790341e-05	64
Верхняя граница диапазона значений входных данных из текста задачи	1.0299991117790341e-05	64

Задача №2. Телефонная книга[N баллов]

В этой задаче ваша цель - реализовать простой менеджер телефонной книги. Он должен уметь обрабатывать следующие типы пользовательских запросов:

- `add number name` – это команда означает, что пользователь добавляет в телефонную книгу человека с именем `name` и номером телефона `number`. Если пользователь с таким номером уже существует, то ваш менеджер должен перезаписать соответствующее имя.
- `del number` – означает, что менеджер должен удалить человека с номером из телефонной книги. Если такого человека нет, то он должен просто игнорировать запрос.
- `find number` – означает, что пользователь ищет человека с номером телефона `number`. Менеджер должен ответить соответствующим именем или строкой «not found» (без кавычек), если такого человека в книге нет.

- **Формат ввода / входного файла (input.txt).** В первой строке входного файла содержится число N ($1 \leq N \leq 10^5$) - количество запросов. Далее следуют N строк, каждая из которых содержит один запрос в формате, описанном выше.

Все номера телефонов состоят из десятичных цифр, в них нет нулей в начале номера, и каждый состоит не более чем из 7 цифр. Все имена представляют собой непустые строки из латинских букв, каждая из которых имеет длину не более 15. Гарантируется при проверке, что не будет человека с именем «not found».

- **Формат вывода / выходного файла (output.txt).** Выведите результат каждого поискового запроса `find` – имя, соответствующее номеру телефона, или «not found» (без кавычек), если в телефонной книге нет человека с таким номером телефона. Выведите по одному результату в каждой строке в том же порядке, как были заданы запросы типа `find` во входных данных.
- Ограничение по времени. 6 сек.
- Ограничение по памяти. 512 мб.
- Примеры:

1:	input.txt	
	12	
	add 911 police	
	add 76213 Mom	
	add 17239 Bob	
	find 76213	
	find 910	
	find 911	
	del 910	
	del 911	
	find 911	
	find 76213	
	add 76213 daddy	
	find 76213	
	output.txt	
	Mom	
	not found	
	police	
	not found	
	Mom	
	daddy	

2:	input.txt	
	8	
	find 3839442	
	add 123456 me	
	add 0 granny	
	find 0	
	find 123456	
	del 0	
	del 0	
	find 0	
	output.txt	
	not found	
	granny	
	me	
	not found	

Описание примера 1. 76213 - это номер Mom, 910 - нет в телефонной книге, 911 - это номер police, но затем он был удален из телефонной книги, поэтому второй поиск 911 вернул «not found». Также обратите внимание, что когда daddy был добавлен с тем же номером телефона 76213, что и номер телефона Mom, имя контакта было переписано, и теперь поиск 76213 возвращает «daddy» вместо «Mom».

```
from lab_6.utils import read_f, write_f

phone_book = {}

def add_(entry):

    phone_number, name = entry.split()

    phone_book[phone_number] = name
```



```

def del_(phone_num) :

    phone_book.pop(phone_num, '')


def do_task(n, a) :

    answer = []

    for i in range(n) :

        command, data = a[i].split(maxsplit=1)

        if command == 'add':

            add_(data)

        elif command == 'del':

            del_(data)

        elif command == 'find':

            answer.append(phone_book.get(data, 'not
found'))

    return answer


if __name__ == "__main__":

    data = read_f(2)

    n = int(data[0])

    data.pop(0)

    res = do_task(n, data)

```

```
write_f(2, *res)
```

Данный код выполняет следующие функции:

Импортирует две функции из модуля `utils`: `read_f` и `write_f`. Эти функции используются для чтения данных из файла и записи результатов в файл соответственно.

Создаёт пустой словарь `phone_book`.

Определяет три функции: `add_`, `del_` и `do_task`. Функция `add_` добавляет запись в словарь `phone_book`, разделяя данные на номер телефона и имя. Функция `del_` удаляет запись из словаря `phone_book` по номеру телефона. Функция `do_task` принимает два аргумента: количество команд `n` и список команд `a`. В этой функции происходит обработка каждой команды. Если команда — «add», то вызывается функция `add_`. Если команда — «del», то вызывается функция `del_`. Если команда — «find», то проверяется наличие номера телефона в словаре `phone_book` и возвращается соответствующее имя или сообщение «not found».

В основной части кода считывается первая строка из файла с помощью функции `read_f`, преобразуется в число `n`, а затем удаляются данные из строки. Далее вызывается функция `do_task` с аргументами `n` и оставшимися данными из файла. Результаты выполнения команд сохраняются в списке `res`. Затем этот список записывается в файл с помощью функции `write_f`.

≡ input.txt × ▾ ⋮

8 ✓
find 3839442
add 123456 me
add 0 granny
find 0
find 123456
del 0
del 0
find 0

≡ output.txt ×

1 not found
2 granny
3 me
4 not found
5

by ≡ input.txt × ▾ ⋮

1 12 ✓
2 add 911 police
3 add 76213 Mom
4 add 17239 Bob
5 find 76213
6 find 910
7 find 911
8 del 910
9 del 911
10 find 911
11 find 76213
12 add 76213 daddy
13 find 76213

≡ output.txt ×

1 Mom
2 not found
3 police
4 not found
5 Mom
6 daddy
7

	Время выполнения	Затраты памяти
--	------------------	----------------

Нижняя граница диапазона значений входных данных из текста задачи	1.3299991117790341e-05	138
Пример из задачи	1.3299991117790341e-05	138
Верхняя граница диапазона значений входных данных из текста задачи	1.4499993994832039e-05	138



Задача №5. Выборы США[N баллов]

5 задача. Выборы в США

Как известно, в США президент выбирается не прямым голосованием, а путем двухуровневого голосования. Сначала проводятся выборы в каждом штате и определяется победитель выборов в данном штате. Затем проводятся государственные выборы: на этих выборах каждый штат имеет определенное число голосов — число выборщиков от этого штата. На практике, все выборщики от штата голосуют в соответствии с результатами голосования внутри штата, то есть на заключительной стадии выборов в голосовании участвуют штаты, имеющие различное число голосов. Вам известно за кого проголосовал каждый штат и сколько голосов было отдано данным штатом. Подведите итоги выборов: для каждого из участника голосования определите число отданных за него голосов.

- **Формат ввода / входного файла (input.txt).** Каждая строка входного файла содержит фамилию кандидата, за которого отдают голоса выборщики этого штата, затем через пробел идет количество выборщиков, отдавших голоса за этого кандидата.
- **Формат вывода / выходного файла (output.txt).** Выведите фамилии всех кандидатов в *лексикографическом* порядке, затем, через пробел, количество отданных за них голосов.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 64 мб.
- Примеры:

№	input.txt	output.txt
1	McCain 10 McCain 5 Obama 9 Obama 8 McCain 1	McCain 16 Obama 17

№	input.txt	output.txt
2	ivanov 100 ivanov 500 ivanov 300 petr 70 tourist 1 tourist 2	ivanov 900 petr 70 tourist 3

№	input.txt	output.txt
3	bur 1	bur 1

```
from lab_6.utils import read_f, write_f

def sort(s):
    answer = sorted(s.items(), key=lambda x: x[0])
    return answer
```

```

def do_task(n, a, s):
    for i in range(n):
        name, count = a[i].split()
        if name in s:
            s[name] += int(count)
        else:
            s[name] = int(count)

    ans = sort(s)
    return ans

if __name__ == "__main__":
    data = read_f(5)
    s = {}
    n = len(data)
    res = do_task(n, data, s)
    write_f(5, *res)

```

Импортирует две функции из модуля `utils`: `read_f` и `write_f`. Эти функции используются для чтения данных из файла и записи результатов в файл соответственно.

Определяет функцию `sort`, которая принимает словарь `s` в качестве аргумента и возвращает отсортированный список кортежей, где каждый кортеж содержит ключ и значение из словаря `s`. Сортировка происходит по ключу.

Определяет функция `do_task`, которая также принимает три аргумента: количество команд `n`, список команд `a` и словарь `s`. В этой функции происходит обработка каждой команды. Если ключ `name` есть в словаре `s`, то значение этого ключа увеличивается на число `count`. Если ключа `name` нет в словаре, то он создаётся со значением `count`.

В основной части кода считывается первая строка из файла с помощью функции `read_f`, преобразуется в число `n`. Затем создаётся пустой словарь `s`. Далее вызывается функция `do_task` с аргументами `n`, данными из файла

data и словарем s. Результаты выполнения команд сохраняются в списке res. Затем этот список записывается в файл с помощью функции write_f.

```

Task_5\...\input.txt
1 McCain 10 ✓
2 McCain 5
3 Obama 9
4 Obama 8
5 McCain 1

output.txt
1 ('McCain', 16)
2 ('Obama', 17)
3

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.12399991117790341e-05	125
Пример из задачи	1.485895959595951e-05	125
Верхняя граница диапазона значений входных данных из текста задачи	1.680000034160912e-05	125

Задача №6. Фибоначчи возвращается [N баллов]

Вам дается последовательность чисел. Для каждого числа определите, является ли оно числом Фибоначчи. Напомним, что числа Фибоначчи определяются, например, так:

$$\begin{aligned} F_0 &= F_1 = 1 \\ F_i &= F_{i-1} + F_{i-2} \text{ для } i \geq 2. \end{aligned} \quad (1)$$

- **Формат ввода / входного файла (input.txt).** Первая строка содержит одно число N ($1 \leq N \leq 10^6$) - количество запросов. Следующие N строк содержат по одному целому числу. При этом соблюдаются следующие ограничения при проверке:
 1. Размер каждого числа не превосходит 5000 цифр в десятичном представлении.
 2. Размер входа не превышает 1 Мб.
- **Формат вывода / выходного файла (output.txt).** Для каждого числа, данного во входном файле, выведите «Yes», если оно является числом Фибоначчи, и «No» в противном случае.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 128 мб. **Внимание:** есть вероятность превышения по памяти, т.к. сами по себе числа Фибоначчи большие. Делайте проверку на память!
- Примеры:

input.txt	output.txt
8	Yes
1	Yes
2	Yes
3	No
4	Yes
5	No
6	No
7	Yes
8	

```
from lab_6.utils import read_f, write_f

def iterativeFib(all_fib_num, n):
    a, b = 0, 1
    all_fib_num.add(a)
    all_fib_num.add(b)
    for i in range(n):
        a, b = b, a + b
        all_fib_num.add(a)
```



```

        all_fib_num.add(b)
    return all_fib_num

def fib_come_back(all_fib_num, n, data):
    answer = []
    for i in range(n):
        if data[i] in all_fib_num:
            answer.append('Yes')
        else:
            answer.append('No')
    return answer

if __name__ == "__main__":
    all_fib_num = set()
    data = read_f(6)
    n = int(data[0])
    data.pop(0)
    data = list(map(int, data))
    all_fib_num = iterativeFib(all_fib_num, 5000)
    result = fib_come_back(all_fib_num, n, data)
    write_f(6, *result)

```

Импортирует две функции из модуля `utils`: `read_f` и `write_f`. Эти функции используются для чтения данных из файла и записи результатов в файл соответственно.

Определяет функцию `iterativeFib`, которая принимает два аргумента: множество чисел Фибоначчи `all_fib_num` и число `n`. В этой функции происходит вычисление первых `n` чисел последовательности Фибоначчи и добавление их в множество `all_fib_num`.

Определяет функция `fib_come_back`, которая также принимает три аргумента: множество чисел Фибоначчи `all_fib_num`, число `n` и список чисел `data`. В этой функции проверяется, есть ли каждое число из списка `data` в множестве `all_fib_num`. Если да, то в список `answer` добавляется «Yes», иначе — «No».

В основной части кода считывается первая строка из файла с помощью функции `read_f`, преобразуется в число `n`, а затем удаляются данные из

строки. Далее создаётся пустое множество `all_fib_num`. Затем вызывается функция `iterativeFib` с аргументами `all_fib_num` и 5000. После этого данные из файла преобразуются в список чисел и вызывается функция `fib_come_back` с аргументами `all_fib_num`, `n` и списком чисел. Результаты выполнения команд сохраняются в списке `res`. Затем этот список записывается в файл с помощью функции `write_f`.

Task_6\...\output.txt		Task_6\...\input.txt	
1	Yes	1	8
2	Yes	2	1
3	Yes	3	2
4	No	4	3
5	Yes	5	4
6	No	6	5
7	No	7	6
8	Yes	8	7
9		9	8
		10	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	0.002640799997607246	127
Пример из задачи	0.002640799997607246	127
Верхняя граница диапазона значений входных данных из текста задачи	1.113640799997607246	127

Задача №8. Фибоначчи возвращается [N баллов]

В данной задаче у Вас не будет проблем ни с вводом, ни с выводом. Просто реализуйте быструю хеш-таблицу.

В этой хеш-таблице будут храниться целые числа из диапазона $[0; 10^{15} - 1]$. Требуется поддерживать добавление числа x и проверку того, есть ли в таблице число x . Числа, с которыми будет работать таблица, генерируются следующим образом. Пусть имеются четыре целых числа N, X, A, B такие что:

- $1 \leq N \leq 10^7$
- $1 \leq X \leq 10^{15}$
- $1 \leq A \leq 10^3$
- $1 \leq B \leq 10^{15}$

Требуется N раз выполнить следующую последовательность операций:

- Если X содержится в таблице, то установить $A \leftarrow (A + A_C) \bmod 10^3$, $B \leftarrow (B + B_C) \bmod 10^{15}$.
- Если X не содержится в таблице, то добавить X в таблицу и установить $A \leftarrow (A + A_D) \bmod 10^3$, $B \leftarrow (B + B_D) \bmod 10^{15}$.
- Установить $X \leftarrow (X \cdot A + B) \bmod 10^{15}$.

Начальные значения X, A и B , а также N, A_C, B_C, A_D и B_D даны во входном файле. Выведите значения X, A и B после окончания работы.

- **Формат входного файла (input.txt).** В первой строке входного файла содержится четыре целых числа N, X, A, B . Во второй строке содержится еще четыре целых числа A_C, B_C, A_D и B_D такие что $0 \leq A_C, A_D < 10^3$, $0 \leq B_C, B_D < 10^{15}$.
- **Формат выходного файла (output.txt).** Выведите значения X, A и B после окончания работы.

- Ограничение по времени. 5 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt
4 0 0 0
1 1 0 0
output.txt
3 1 1

```
from lab_6.utils import read_f, write_f
class HashSet:
    EMPTY = -1
    DELETED = -2

    def __init__(self, capacity):
        self.capacity = capacity
        self.htable = [self.EMPTY] * capacity
        self.size = 0
```

```

def find_index(self, x):
    def iter(i, h, place_to_insert=-1):
        if i <= self.size + 1:
            if self.htable[h] == self.EMPTY:
                return (False, place_to_insert if
place_to_insert >= 0 else h)
            elif self.htable[h] == self.DELETED:
                return iter(i + 1, (h + 41) %
self.capacity, place_to_insert if place_to_insert >=
0 else h)
            elif self.htable[h] == x:
                return (True, h)
            else:
                return iter(i + 1, (h + 41) %
self.capacity, place_to_insert)
        else:
            return (False, place_to_insert)

    return iter(0, x % self.capacity)

def insert(self, x):
    found, h = self.find_index(x)
    if not found:
        self.size += 1
        self.htable[h] = x
        return True
    return False

def main(data):
    a1 = 1000
    b1 = 10**15

    n, x, a, b = map(int, data[0].split())
    ac, bc, ad, bd = map(int, data[1].split())

    set_hash = HashSet(15000017)
    for _ in range(n):
        if not set_hash.insert(x):

```

```

        a = (a + ac) % al
        b = (b + bc) % bl
    else:
        a = (a + ad) % al
        b = (b + bd) % bl
    x = (x * a + b) % bl

    return x, a, b

if __name__ == "__main__":
    data = read_f(8)
    res = main(data)
    s = ''
    for i in range(3):
        s+=str(res[i])+ ' '
    write_f(8, s)

```

Импортируются две функции из модуля `utils`: `read_f` и `write_f`. Эти функции используются для чтения данных из файла и записи результатов в файл соответственно.

Определяется класс `HashSet` с конструктором `init`, который принимает один аргумент — `capacity`. В конструкторе создаётся таблица `htable` заданного размера, которая будет использоваться для хранения элементов, и устанавливается размер таблицы `size` равным нулю.

Определяется метод `find_index`, который ищет индекс элемента в таблице. Метод принимает один параметр — элемент `x`, который нужно найти. Если элемент найден, то метод возвращает `True` и индекс элемента, иначе — `False` и значение по умолчанию.

Определяется метод `insert`, который добавляет элемент в таблицу. Сначала вызывается метод `find_index` для поиска индекса элемента. Если индекс не найден, то элемент добавляется в таблицу по найденному индексу.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений	0.025279299999632761	198

ВХОДНЫХ ДАННЫХ ИЗ ТЕКСТА ЗАДАЧИ		
Пример из задачи	0.02527929999632761	198
Верхняя граница диапазона значений входных данных из текста задачи	0.02527929999632761	198

Вывод

В данной лабораторной работе я ознакомилась с такими понятиями, как множество, словари, хеш-таблицы и хеш-функции.